# En kortfattet introduktion til uv

*uv – et lynhurtigt værktøj, der gør dit liv med Python lettere.*

Som du måske ved, er det vigtigt at have styr på dine virtuelle miljøer, når du arbejder i Python. Virtuelle miljøer er som regel defineret i en mappe, der hedder `.venv`.

`uv` er et værktøj, der styrer Python-versioner, virtuelle miljøer og moduler i ét. Det erstatter `pip`, `venv` og `requirements.txt`.

Officiel dokumentation: docs.astral.sh/uv

---

## Installér uv

Hent uv. Gå til: Installing uv

Følg installationsvejledningen.

Når du er færdig, så tjek, at det virker i terminalen.

```
uv --version
```

Output (eksempel):

```
uv 0.12.22 (70fe1196a 2026-10-01 aarch64-apple-darwin)
```

---

## Hurtig start

Sådan ser hele forløbet ud, fra tomt projekt til kørende program. Kommandoerne forklares i de næste afsnit.

```
uv init --no-package mit-projekt   # opret projektet
cd mit-projekt                     # gå ind i mappen
uv add flask                       # tilføj et modul
uv run main.py                     # kør programmet
```

Output:

```
Hello from mit-projekt!
```

---

## Opret et projekt: `uv init`

Du opretter et nyt projekt med `uv init`.

Hvis din projektmappe eksisterer i forvejen, kan du udelade navnet og blot skrive `uv init --no-package`.

```
cd min-mappe            # skift til din mappe
uv init --no-package
```

Når du opretter et projekt, kan du vælge mellem flere skabeloner, fx applikation og bibliotek.

Her opretter vi et nyt projekt i en ny mappe.

```
uv init --no-package mit-projekt   # med navn
cd mit-projekt
```

Det giver denne mappestruktur:

```
mit-projekt/
├── .gitignore
├── .python-version
├── main.py
├── pyproject.toml
└── README.md
```

Kommandoen er altså `uv init --no-package`, fordi vi ikke vil oprette en pakke (package) med src-mappe i første omgang. Brug denne til de fleste opgaver og små programmer.

Se mere i dokumentationen: Working on projects.

### Vælg Python-version

Du kan også vælge en bestemt Python-version, når du opretter projektet.

```
uv init --python 3.12 mit-projekt
```

### Se projektfilerne

Disse filer beskriver projektet og skal med i Git:

| Fil | Indhold |
|---|---|
| `pyproject.toml` | Projektets navn, Python-version og moduler |
| `uv.lock` | Præcise versioner af alle moduler |
| `.python-version` | Python-versionen projektet bruger |

Eksempel på `pyproject.toml`:

```toml
[project]
name = "mit-projekt"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "flask>=3.1.0",
    "requests>=2.32.3",
]
```

---

## Tilføj moduler: `uv add`

Du tilføjer og fjerner moduler med `uv add` og `uv remove`.

```
uv add requests
uv add requests flask        # flere på én gang
uv add "flask>=3.0"          # med krav til version
```

Fjern et modul:

```
uv remove flask
```

Moduler kun til udvikling (fx test):

```
uv add --dev pytest
```

`uv add` opdaterer automatisk `pyproject.toml` og `uv.lock` og installerer modulet i `.venv`.

> Brug **ikke** `pip install` i et uv-projekt. Brug `uv add`.

---

## Kør dine scripts: `uv run`

Når du bruger `uv` til at håndtere Python, kører du dine programmer med `uv run`.

```
uv run main.py
uv run app.py
uv run python                # interaktiv Python i projektets miljø
```

`uv run` sørger selv for at miljøet er opdateret, før filen køres. Du behøver ikke aktivere `.venv`.

Første gang du kører `uv run` (eller `uv add` eller `uv sync`), oprettes mappen med dit virtuelle miljø (`.venv`).

**VS Code:** Vælg fortolkeren `.venv` med *Python: Select Interpreter*. Så virker ▶-knappen også.

---

## Genskab et projekt: `uv sync`

Her kloner du et projekt fra GitHub.

```
git clone https://github.com/<bruger>/<projekt>.git
cd <projekt>
```

Derefter kan du synkronisere projektet lokalt.

```
uv sync
```

`uv sync` læser `pyproject.toml` og `uv.lock` og bygger `.venv` op med præcis de samme versioner.

---

## Oversigt

| Opgave | Kommando |
|---|---|
| Nyt projekt (uden src) | `uv init --no-package navn` |
| Nyt projekt med src (pakke) | `uv init navn` |
| Tilføj modul | `uv add modul` |
| Fjern modul | `uv remove modul` |
| Kør fil | `uv run fil.py` |
| Vis moduler | `uv pip list` / `uv tree` |
| Vis Python-versioner | `uv python list` |
| Genskab miljø | `uv sync` |
| Opdater lock-fil | `uv lock` |
| Opdater uv | `uv self update` |